

Lab 4: Vibe Coding – Minesweeper

Name: Anvii S Rai
SRN: PES2UG24CS076
SECTION: 5B

CHAT HISTORY

Objective

Use an LLM as a coding assistant to inspect the existing Minesweeper project, identify defects, implement the required features, and test the final solution.

Task 1 – Boundary-Safe Board Traversal

Initial Analysis

The project was first executed using:

```
python main.py
```

The game launched successfully and displayed a 6×6 Minesweeper board.

The `board.py` file was inspected to understand how neighbor traversal and flood-fill expansion were implemented.

The following code was identified in the `neighbors()` function:

```
if 0 <= nr <= self.rows and 0 <= nc <= self.cols:  
    yield nr, nc
```

Defect Identified

The condition incorrectly allowed coordinates equal to `self.rows` and `self.cols`.

For a 6×6 board:

Valid rows: 0–5

Valid columns: 0–5

However, the existing code also allowed:

Row 6

Column 6

which are outside the board boundaries.

This could cause invalid neighbor traversal during mine counting and reveal expansion.

Fix Applied

The condition was changed to:

```
if 0 <= nr < self.rows and 0 <= nc < self.cols:  
    yield nr, nc
```

Verification

The game was tested using corner and edge reveals:

```
r 1 6  
r 6 1  
r 6 6
```

Results:

- No crashes occurred.
- Reveal expansion remained inside the board.
- Flood-fill behaved correctly.

Task 1 completed successfully.

Task 2 – Win and Flag Behaviour

Code Review

The reveal logic was reviewed:

```
if pos in self.revealed or pos in self.flags:  
    continue
```

This prevents flagged cells from being revealed.

The win condition was reviewed:

```
def won(self):  
    return len(self.revealed) == self.rows * self.cols - self.mine_total
```

This correctly detects victory only when all non-mine cells have been revealed.

Testing

The following commands were executed:

```
f 1 1  
r 1 1  
f 1 1
```

Observed behaviour:

- Flag appeared correctly.
- Revealing a flagged cell had no effect.
- Flag could be removed successfully.

Task 2 completed successfully.

Task 3 – Difficulty Modes

Requirement

Add Easy, Medium, and Hard modes without introducing additional files or persistent storage.

Implementation

The constructor was updated to support configurable board dimensions and mine counts:

```
def __init__(self, rows=6, cols=6, mines=6):  
    self.board = Board(rows=rows, cols=cols, mines=mines)
```

A difficulty-selection menu was added:

```
print("Select difficulty:")  
print("1. Easy (6x6, 6 mines)")  
print("2. Medium (8x8, 10 mines)")  
print("3. Hard (10x10, 15 mines)")
```

The selected mode creates an appropriate board in memory.

Testing

Easy

1

Result:

6 × 6 board

Medium

2

Result:

8 × 8 board

Hard

3

Result:

10 × 10 board

Task 3 completed successfully.

Task 4 – Action-Level Feedback

Requirement

Provide feedback for a player's reveal action while ensuring that internal flood-fill expansion does not generate repeated messages.

Implementation

Added:

```
print(f"Revealed ({r + 1}, {c + 1})")
```

after successful reveal commands.

Testing

Command:

```
r 4 4
```

Output:

```
Revealed (4, 4)
```

The board updated correctly.

Large reveal expansions still produced only a single feedback message because the message is tied to the player's command rather than internal flood-fill iterations.

Task 4 completed successfully.

Final Testing

The following cases were tested:

- Corner reveals
- Edge reveals
- Center reveals
- Zero-adjacent flood-fill regions
- Mine hits
- Flag toggling
- Flag protection
- Invalid commands
- Invalid coordinates
- Easy difficulty
- Medium difficulty
- Hard difficulty
- Action-level feedback

All required tasks were completed successfully.

Summary of Changes

1. Fixed boundary traversal in neighbor generation.
2. Verified correct win-condition behaviour.
3. Verified flag protection and toggling.
4. Added Easy, Medium, and Hard difficulty modes.
5. Added player-facing reveal feedback.
6. Tested all required edge cases and gameplay scenarios.

Representative Prompts Used During Development

1. Inspect the Minesweeper code and identify the Task 1 defect related to board traversal.
2. Explain how the neighbor traversal logic works and why the current implementation can generate invalid coordinates.
3. Suggest a minimal fix for the boundary traversal bug without changing the overall project structure.
4. Review the flag and win-condition logic and verify whether it satisfies Task 2 requirements.
5. Add Easy, Medium, and Hard difficulty modes while keeping all state in memory.
6. Add action-level feedback for reveal commands without generating repeated messages during flood-fill expansion.
7. Suggest test cases for corners, edges, invalid inputs, flags, mine hits, and difficulty modes.
8. Review the final implementation against the README checklist.